SAARTHI: An AI-Powered Industrial Safety Copilot for Real-Time Excavator Hazard Detection

Sounak Chowdhury, Noel Binu, Soham Chakraborty, Sharvari Santosh Mahurkar, Onkar Chalekar, and Kumaran K

School of Computer Science and Engineering, Vellore Institute of Technology, Chennai 600127, India
Corresponding author: Kumaran K (kumaran.k@vit.ac.in)

***Abstract*—Keeping heavy machinery operators safe is difficult because most collision and fatigue monitors work in total isolation. A cabin camera doesn't know what is in the vehicle's blind spot, and proximity sensors cannot tell if a driver is falling asleep. Because these older systems rely on fixed, hard-coded rules, they generate constant false alarms on busy worksites. Eventually, operators just stop paying attention to them altogether. We built SAARTHI to solve this disconnect. It is a personalized AI copilot that fuses different safety feeds into a single Hybrid Decision Engine. The system actively combines spatial hazard tracking via YOLO, physiological data from MediaPipe, and raw machine telemetry. To evaluate the actual danger of a situation, an XGBoost-driven Contextual Risk Engine adjusts the baseline score based on weather, shift timing, and operator experience. Instead of relying on a static siren, intervention decisions are handled by a Proximal Policy Optimization (PPO) reinforcement learning model. The agent learns exactly when and how to alert the operator without causing alarm fatigue. We also built the interface to adapt automatically to the person in the seat, swapping standard audio for haptic feedback or high-contrast visuals based on their specific profile. Hosted on an asynchronous FastAPI backend, experimental results demonstrate that SAARTHI achieves an F1-Score of 0.952 and an AUC of 0.932 on heavily imbalanced datasets. The architecture proves that multi-modal, highly personalized safety constraints can run in real time across industrial fleets without lagging, maintaining an average inference latency of 0.55 ms and a minimal memory footprint of 165.75 KB.**

***Index Terms*—Industrial Safety, Computer Vision, YOLOv11, Excavator, Hazard Detection, Edge AI, Large Language Models.**

I. Introduction

The construction and industrial manufacturing sectors are highly hazardous environments where rigorous safety monitoring is critical to preventing accidents. Workers constantly operate in close proximity to heavy machinery and dynamic spatial obstacles, making consistent and safe reactions to traffic and machinery hazards a major challenge. A significant, yet often overlooked, issue in modern safety monitoring is alarm fatigue; repeated exposure to static or continuous alarm patterns causes workers to become desensitized, thereby drastically reducing their vigilance and responsiveness to actual hazards over time [1].

Operator impairment, specifically drowsiness, further compromises worksite safety. Drowsiness induced by sleep deprivation results in microsleeping and prolonged eyelid closures, both of which are critical early indicators of fatigue that lead to fatal misjudgments. Modern computer vision systems have proven highly effective at identifying these states by monitoring facial landmarks to compute the percentage of eyelid closure and estimating three-dimensional head poses [2]. Concurrently, deep learning architectures like the You Only Look Once (YOLO) algorithm are widely utilized to rapidly and accurately detect external targets—such as workers and personal protective equipment—within complex, heavily occluded industrial scenes [3].

However, the reality is that worksite hazards are compound events, whereas existing safety systems operate in single-modality silos. A cabin camera capable of tracking a driver's facial landmarks remains entirely unaware of an external worker standing in the machine's blind spot. Conversely, a robust YOLO-based object detector cannot determine if the machine operator is falling asleep. When human operators and supervisors are forced to mentally fuse these independent alerts on the fly, reaction times plummet. Furthermore, addressing alarm desensitization requires non-stationary, adaptive control rather than fixed thresholds. Reinforcement Learning (RL) has shown significant promise in this domain, as RL agents can dynamically adjust alarm attributes based on specific reward functions to ensure safe and consistent hazard reactions [1].

To integrate these disjointed technologies into a cohesive framework, this paper introduces Saarthi: a personalized, real-time AI safety copilot built for heavy-machinery operators facing multi-hazard risks. By fusing YOLO-based external hazard detection, facial landmark-based fatigue monitoring, and RL-driven adaptive interventions into a centralized Hybrid Decision Engine, the system anticipates and mitigates accidents proactively.

A. Novelty

The novelty of the proposed Saarthi system in terms of adaptive, multi-modal industrial safety is presented as follows:

- A **Hybrid Decision Engine**: Fuses external visual intelligence via YOLO models for spatial object detection with internal physiological monitoring using facial landmarks and eye closure metrics. This linkage catches complex, compound hazards that standalone detectors completely miss.
- A dynamic **Contextual Risk Engine**: Adjusts baseline safety scores using a trained XGBoost Regressor. It factors in real-world operational variables like shift timing, weather, machine type, and the operator's specific experience level.
- An **Adaptive Intervention Logic**: Powered by a Proximal Policy Optimization (PPO) Reinforcement Learning model. Because fixed rules for alarm delivery fail in non-stationary environments and directly contribute to desensitization, this RL agent learns the optimal timing and intensity of interventions to prevent alarm fatigue.
- An **accessibility-first User Interface (UI)**: Dynamically shifts alert modalities based on operator profiles. It swaps standard audio alarms for haptic overlays (for hearing-impaired users) or high-contrast visuals, ensuring personalized safety delivery without compromising the underlying RL safety constraints.

B. Contribution

These contributions to multi-hazard operator safety and fatigue monitoring are as follows:

- **Multi-modal risk fusion**: We engineered a concurrent processing pipeline that seamlessly aggregates high-frequency facial landmark tracking, real-time object detection, and vehicle telemetry into a single, cohesive safety score.
- **Balancing safety and personalization**: We deployed a constrained RL policy that dynamically adjusts alarm attributes—combating alarm fatigue while preserving safe worker reactions—by enforcing strict safety boundaries against individual operator profiles.
- **Scalable and accessible deployment**: We built a high-performance, asynchronous FastAPI backend capable of handling intense parallel CPU loads without lagging the operator's real-time dashboard, demonstrating that advanced AI safety tools can scale inclusively across large industrial fleets.

II. Literature Review

Industrial safety management has increasingly relied on artificial intelligence to reduce the frequency of operator-related accidents on heavy machinery and construction sites [4]. Fatigue detection has received considerable attention in this area, where facial landmark analysis is used to derive indicators such as eye aspect ratio (EAR), percentage of eyelid closure over time (PERCLOS), and head pose, which are subsequently classified using machine learning models such as XGBoost or support vector machines [5]. These approaches are generally effective at identifying drowsiness from behavioral cues, although their reliability depends heavily on lighting conditions and camera placement inside the cabin.

Alongside fatigue monitoring, vision-based object detection has been applied for blind-spot and worker-proximity awareness around excavators, loaders, and similar equipment. Architectures from the YOLO family are commonly used for this purpose, with detected objects mapped onto predefined danger zones around the machine to trigger proximity alerts [6]. Separately, a body of work has used historical incident data, including OSHA-reported injury records, to train predictive models for contextual risk estimation. These models typically use gradient-boosted classifiers to relate variables such as machine type, task category, weather, and time of day to accident likelihood, producing a risk score that supports supervisory decision-making rather than real-time intervention [7].

Reinforcement learning has also been explored for adaptive decision-making in safety-related systems, most notably through policy-gradient methods such as Proximal Policy Optimization (PPO), trained within simulated environments to learn intervention behavior rather than relying on manually specified rules [8]. However, the majority of this work has been directed at autonomous vehicle or robotic control problems, with limited application to systems that must warn a human operator rather than control the machine directly. In parallel, explainability techniques such as SHAP have gained traction in safety-critical domains, where they are used to attribute model outputs to the underlying input features and support operator or supervisor trust in automated alerts [9].

Taken together, these lines of work show that fatigue detection, proximity-based hazard detection, contextual risk estimation, and reinforcement-learning-based decision-making have each reached a reasonable level of maturity when studied independently. What is comparatively underexplored is their combination into a single system capable of reasoning jointly about operator condition, external hazards, and machine state, particularly one that also accounts for how the resulting alert should be delivered to a specific operator.

A. Limitations and Research Gap

While the reviewed techniques have individually contributed to improved hazard detection and risk estimation, several limitations remain when these systems are considered as a whole. Most existing systems are designed around a single hazard category, so a given deployment typically handles either fatigue monitoring or proximity detection, but rarely both together with additional factors such as machine tilt or stability. Where intervention logic is present, it is usually governed by fixed thresholds that do not adjust to individual operator behavior or to the operating context, which can result in either delayed warnings or an excess of false alerts. Although reinforcement learning has been used for adaptive decision-making in related domains, most reported implementations do not include an explicit mechanism to guarantee that a learned policy cannot under-respond to a genuinely critical situation, since safety behavior is implicit in the reward formulation rather than structurally enforced. Explainability, similarly, is usually applied during offline model evaluation rather than surfaced to the end user at the moment an alert is triggered, which limits an operator's ability to trust or verify the decision in real time.

A further limitation concerns personalization of alert delivery. Existing systems generally assume a uniform operator profile, with alert modality fixed at design time regardless of hearing or visual impairment, experience level, or language, despite this directly affecting whether a warning is perceived and understood in time. Where multiple AI modules are deployed together, they typically operate as separate pipelines rather than a coordinated decision system, leaving the task of synthesizing several alert streams to a human supervisor. Finally, a substantial portion of the reviewed literature assumes network or cloud connectivity for inference, which is not a realistic assumption for remote worksites and heavy-machinery cabs.

These gaps indicate a clear need for an integrated, priority-aware safety framework capable of combining multi-modal hazard sensing, a hybrid rule-based and learning-based decision architecture, accessibility-driven personalization, and real-time explainability, while operating fully offline. To address these gaps, this work proposes SAARTHI, a unified safety copilot that fuses fatigue detection, blind-spot monitoring, and machine tilt sensing with a two-tier decision engine — deterministic hard rules for known critical hazards, combined with a PPO-trained adaptive policy for softer intervention levels — together with accessibility-aware alert personalization and reason-based explainability, deployed entirely on-device.

III. Related Work

Securing heavy-machinery operations sits at the intersection of computer vision, physiological monitoring, and adaptive decision-making. Broader surveys of industrial safety still tend to focus on rule-based alarms and single-sensor approaches, and even where hybrid deep learning architectures have made real progress on driver drowsiness detection in automotive settings, they are built around a single modality. That assumption breaks down on a construction site, a mine, or a factory floor, where operators differ widely in experience and ability and hazards rarely show up one at a time. This section works through that prior literature and situates the design choices behind SAARTHI against it.

A. Threshold-Based and Infrastructure-Centric Safety Systems

Most collision-prevention systems on active worksites still come down to proximity alarms and geofencing: fixed ultrasonic or radar sensors mounted on the machine, scanning continuously and comparing whatever they detect against a static, manually set threshold, with no regard for who's actually operating the machine. In practice, those fixed thresholds tend to misfire constantly on busy sites, where workers and equipment are always moving through the sensor's range. Worse, the same threshold gets applied to every operator regardless of experience or physical ability, so an experienced operator ends up tuning out the alarms while a newer one may not get warned early enough.

Newer cabin-integrated systems have brought computer vision into the mix, using object detection to pick out workers, vehicles, and structural hazards around the machine, and drawing on signals like facial landmark drift, eyelid closure, or chassis tilt to flag unsafe states. But most of these still work in isolation. A camera watching for fatigue has no idea there's a person standing in the machine's blind spot, and a proximity sensor has no way of knowing the operator is falling asleep. Because these hazards arise from genuinely different physical processes, no single detector is well positioned to catch all of them—which is really the argument for fusing several lightweight signals rather than trying to build one model that does everything.

Conventional alarm-based systems, built for fairly controlled, homogeneous environments, don't hold up well once the operator population and the worksite itself become this varied.

- **Rigidity and operator homogeneity.** A fixed-threshold system treats every operator identically, regardless of experience, sensory ability, or fatigue tolerance. In practice that means the same threshold that produces alarm fatigue in a veteran operator will under-warn someone newer to the job—there's no way to individualize the safeguard without changing the underlying logic entirely.
- **Deployment complexity and accessibility gaps.** Combining vision, physiological, and environmental sensing into one cabin system is already a nontrivial integration problem, and most existing architectures weren't designed with that convergence in mind. Accessibility tends to be an afterthought too: operators with hearing or visual impairments are rarely considered when alert delivery is designed almost exclusively around audio cues.

B. Fatigue-Centric and Vision-Based Defenses

A separate line of work moves detection logic onto the operator's own physiological state rather than the surrounding environment. Here the typical approach is a camera-based agent watching the driver's face—tracking eye-aspect-ratio, yawning frequency, head pose—to infer drowsiness before it becomes dangerous.

These systems are reasonably good at what they do, but that's also their limitation: they know almost nothing about what's happening outside the cabin. A fatigue detector can't tell you there's a worker nearby, or that the terrain has become unstable, or that the machine itself is tilting toward a rollover. It's built to catch one kind of hazard and stays blind to the rest.

- **Modality opacity.** Vision-only systems generally ignore machine-state telemetry altogether, even though rollover risk from chassis inclination is well documented in the tilt-sensing literature. A fatigue model has no access to that data by design, so it simply can't detect a compound hazard—drowsiness plus instability, say—even when both are present at once.
- **The context gap.** These tools are built around a single-hazard focus and stay that way in deployment. They fire only on drowsiness cues, missing nearby obstacles or unstable ground entirely. By the time a purely fatigue-driven alert goes off, the operator may already be in a situation involving more than just tiredness.

C. Machine Learning in Industrial Safety

As static, rule-based systems have repeatedly failed to keep up with a workforce this varied in experience and accessibility needs, machine learning has become the default approach for adaptive intervention.

1) Deep Learning and Object Detection Models: Deep learning dominates recent work in this space. YOLO-based detectors, for instance, perform well in real time for identifying blind-spot obstacles and nearby workers. On the physiological side, convolutional facial-landmark models track eye and head movement continuously to catch fatigue before it becomes critical, and gradient-boosted models like XGBoost have proven useful for context-aware risk scoring, weighing environmental and operational variables against historical incident records.

The catch is that these models are almost always deployed as separate pipelines. Nobody is fusing their outputs into one coherent decision—which leaves a human supervisor to piece together several unrelated alerts in real time.

- **Fragmented decision-making.** Running fatigue, obstacle, and tilt detection as three independent systems means a supervisor has to mentally combine uncorrelated alerts on the fly. That adds real cognitive load, slows reaction time, and raises the odds that a compound hazard slips through unnoticed.
- **Static intervention policy.** Even where detection is sophisticated, the response usually isn't: cabins tend to issue the same alert regardless of context, which is exactly the kind of rigid logic an adaptive, constraint-respecting policy is meant to replace.

2) Lightweight Rule-Based Models: Some practitioners have gone the other direction, relying on fixed-weight scoring rules or manually curated alert matrices instead. These are cheap to implement and integrate easily into legacy cabin electronics, which is a real advantage.

The tradeoff is that manual thresholds don't adapt well to individual variation—different operator experience levels, different accessibility needs, changing site conditions. Keeping thresholds current across a large fleet turns into an ongoing manual calibration problem, which doesn't scale. Reinforcement learning offers a way around this, letting a system learn its own intervention policy while staying inside predefined safety bounds, though getting RL to behave safely and explainably in a live industrial setting is still an open problem—earlier work has shown that unconstrained policy learners can behave unpredictably during exploration, especially when safety constraints aren't built into training from the start.

D. The Gap Analysis

Taken together, the literature points to a fairly clear gap: nobody has really combined multiple hazard-detection modalities, adaptive decision-making, explainability, and operator-specific accessibility into one system. Most work picks one of these and pushes it further, while the others—particularly personalization—stay largely unaddressed.

SAARTHI is built around closing that specific gap. It fuses fatigue, blind-spot, tilt, and contextual risk signals into a single Reinforcement Learning policy operating under safety constraints, giving it the kind of adaptive intervention selection that has mostly been left to human judgment after the fact, while keeping the decision process explainable enough for safety-critical use. Rather than treating accessibility as something bolted on afterward, operator-specific delivery preferences are built directly into the Hybrid Decision Engine—so the same underlying risk score can reach an operator through haptic, visual, or audio channels depending on what they actually need, without changing the safety guarantees behind the decision itself.

IV. System Model and Theoretical Framework

To ground the design in real operating conditions, we first characterize the driver and the machine's surrounding environment as they evolve in real time. On-device deployment inside a moving vehicle comes with hard limits—a tight latency budget and modest compute—and these constraints shape everything that follows. They rule out streaming raw, uncompressed HD frames, and they rule out running heavy spatio-temporal deep networks directly at the edge, which is exactly where traditional vehicle-tracking pipelines fall short.

SAARTHI takes a different route. It represents the operator and machine state through localized vision primitives paired with synchronized telemetry. This approach compresses the high-frequency video and sensor streams into a compact risk feature vector while preserving the information that actually matters for hazard detection: the signatures of micro-sleep, blind-spot intrusions, and axis tilt. The sections below lay out the hazard threat model, the reasoning behind real-time risk estimation, and the localized telemetry features used to flag imminent operational anomalies.

TABLE I
Comparative Analysis of Operator Safety Paradigms

Methodology	Hardware Class	Latency	Primary Limitation
Cloud-Based Telematics	Server + Cellular Node	High	Connectivity reliance in remote sites (e.g. mines)
Wearable Biosensors	Smartwatch / EEG Cap	Low	Operator discomfort, hygiene, compliance drop-off
Heavy Deep Learning (CNN/ViT)	Edge GPU (Jetson/TPU)	Medium	High computational complexity, thermal output
Basic Machine Thresholds	Machine ECU	Low	Rigidly generic, no behavioral insight
SAARTHI (Proposed)	Commodity Edge CPU	Low	Requires 30 s initial operator calibration

A. The Hazard Model

Hazards in industrial vehicle environments fall into two broad classes, separated by how visible they are and how much computational reasoning reliable detection demands.

1) Type 1: Overt Operational Hazards: A Type 1 hazard occurs whenever a physical safety boundary or operational constraint is breached—a pedestrian stepping into a blind spot, an operator letting go of the wheel with both hands, or any incursion into a predefined safety zone. Like conventional rule-based anomaly detection, these events carry clear, observable visual signatures, which makes them deterministic and comparatively straightforward to catch.

SAARTHI detects them in real time by checking the bounding boxes produced by the YOLO detector against predefined restricted regions. The moment an object crosses into a prohibited zone, the system flags a Type 1 hazard and triggers the appropriate safety response.

2) Type 2: Covert Micro-behavioral Hazards: Type 2 hazards are subtle behavioural or contextual anomalies, and they are far harder to spot. Because they tend to blend into normal operation, simple spatial rules or threshold-based vision cannot isolate them. An operator in the early stages of fatigue, for instance, may sit with an apparently normal posture and open eyes while slowly drifting into reduced alertness. Machine vibration or tilt behaves the same way: each reading may stay within acceptable bounds on its own, yet together the readings point to an abnormal operating state.

Since these situations look almost identical to a safe state on the surface, vision alone is not enough. SAARTHI instead examines the statistical character and temporal evolution of the operator's micro-movements—blink-rate variation, gaze drift, head pose—alongside machine telemetry over time. This deeper behavioural view surfaces latent hazards before they escalate into critical incidents.

B. Anomaly Theory in the Physiological and Kinematic Sense

We monitor the operator's state in real time using facial-landmark coordinate features. The core primitive is the Eye Aspect Ratio (EAR), a per-frame measure of how open the eyes are:

$$EAR = \frac{||p_2 - p_6|| + ||p_3 - p_5||}{2||p_1 - p_4||} \tag{1}$$

Here p_i denotes the 2D position of an eye landmark.

For an alert operator, the EAR sequence stays fairly stable around an open-eye mean μ_{alert}, and the fluctuations that do occur—ordinary physiological blinks, small head-pose shifts, minor lighting changes—sit within a natural range. The resulting eye-state distribution is therefore roughly bell-shaped and symmetric.

The framework rests on a simple premise: the onset of fatigue introduces specific, non-Gaussian distortions into this physiological stream.

- **Microsleep artifacts.** Prolonged eye closures—the hallmark of severe drowsiness—extend well beyond the normal blink window. They appear as rapid collapses in the EAR stream, the phenomenon the PERCLOS metric is designed to quantify, and they look nothing like the smooth, brief dip of an ordinary blink.
- **Heavy-tailed distributions.** These long closures skew the eye-state distribution and thicken its lower tail. As the operator jerks awake and tries to recover the open-eye mean μ, the distribution no longer looks symmetric—its heavier tail is the statistical fingerprint of fatigue.

Kinematic anomalies—rollover risk, blind-spot intrusion—are handled on a separate track, driven by machine telemetry and spatial bounding boxes. When the machine is moving ($v \neq 0$) and the tilt angle θ exceeds a critical set-point θ_{crit}, the response is deterministic rather than behavioural: a Tier-1 safety intervention fires immediately, independent of the standard behavioural policies.

C. Mathematical Formalization of Features

To turn these physiological cues into something a classifier can act on, the system extracts a feature vector $\mathbf{x}$ from a sliding temporal window $W = \{e_1, e_2, \dots, e_w\}$ of raw EAR samples. Table II lists the notation used throughout the derivation.

TABLE II
MATHEMATICAL NOTATIONS AND DEFINITIONS. *These symbols define the statistical moments used to construct the feature vector* $\mathbf{x}$*, mapping raw physiological data into the decision space.*

Symbol	Definition
W	Sliding temporal window of size w
e_i	Individual EAR sample at frame i
μ	Mean EAR (first moment)
σ	Standard deviation (second moment)
S_k	Skewness (third standardized moment)
K_u	Excess kurtosis (fourth standardized moment)
E_{roc}	EAR rate of change
$\Phi(\mathbf{x})$	Non-linear mapping to the XGBoost feature space

1) Skewness (S_k): Skewness measures the asymmetry of the signal's distribution. For an alert operator the eye-openness distribution stays close to symmetric over time ($S_k \approx 0$). As fatigue sets in, micro-sleeps and long closures pull the EAR values toward the low, closed-eye end, producing an asymmetric lower tail.

$$S_k = \frac{\frac{1}{w}\sum_{i=1}^{w}(e_i - \mu)^3}{\left(\frac{1}{w}\sum_{i=1}^{w}(e_i - \mu)^2\right)^{3/2}} \tag{2}$$

2) Kurtosis (K_u): Kurtosis is the framework's primary discriminator for deep drowsiness. It measures the tailedness of the distribution:

$$K_u = \frac{\frac{1}{w}\sum_{i=1}^{w}(e_i - \mu)^4}{\left(\frac{1}{w}\sum_{i=1}^{w}(e_i - \mu)^2\right)^{2}} - 3 \tag{3}$$

Subtracting 3 yields excess kurtosis, normalized against a standard normal distribution. Values far from 0 flag the kind of outliers consistent with microsleep episodes rather than ordinary blinking.

3) EAR Rate of Change (E_{roc}): The EAR rate of change captures the fast transitions typical of a fatigue episode—a sudden jerk awake, a rapid head drop:

$$|E_{roc}| = |e_t - e_{t-k}| \tag{4}$$

Here $k = 5$ sets the stride, measuring displacement across several frames. A large $|E_{roc}|$ signals a rapid eye snap or an abrupt physiological shift—motion you rarely see in calm, alert operation, where the state drifts slowly.

D. Defining "Real-Time" for Edge AI Safety

One of the first and crucial steps in Edge AI Safety is to clearly define what is meant by "Real-Time". However, the statistical physiological analysis requires a minimum number of w frames to be able to extract a meaningful distribution to detect fatigue.

This paper defines Real-Time in the context of Operator Safety as:

$$T_{\text{inference}} \ll T_{\text{feature\_window}} < T_{\text{hazard\_onset}} \tag{5}$$

TABLE III
SENSITIVITY ANALYSIS OF THE FEATURES. *Each feature maps to a particular type of anomaly. Mean EAR supplies physiological background, while kurtosis and skewness pick up the non-Gaussian behaviour that characterizes the onset of fatigue.*

Feature	Physiological Meaning	Detection Function
Mean EAR (μ)	Baseline openness	Establishes the EAR baseline, providing physiological context.
Std. dev. (σ)	Blink volatility	Stability check: high variance flags erratic blinking or an unstable head pose.
Kurtosis (K_u)	Drowsiness burstiness	Primary discriminator: detects deep micro-sleeps; longer closures raise kurtosis.
Skewness (S_k)	Distribution symmetry	Fatigue artifacts: detects the closed-eye asymmetry in the distribution.
Rate of change (E_{roc})	Physiological velocity	Flags rapid eye movements or sudden head drops.

The data collection time ($T_{\text{feature\_window}}$) is not the critical metric, but rather the Inference Latency ($T_{\text{inference}}$) is the critical metric for Real-Time in the context of Operator Safety. The continuous feature extraction window is implemented with a moving buffer. When a Critical Fatigue event occurs, or when an operator shows signs of Critical Fatigue, a verdict should be available from the Hybrid Decision Engine ($T_{\text{hazard\_onset}}$). With a considerably lower algorithmic processing time for the inference (milliseconds) compared to the time it takes the human to react in order to prevent an accident (T_{human}), the system can issue a customised warning earlier. To avoid this decision logic becoming the performance obstacle, SAARTHI will reduce $T_{\text{inference}}$ by using lightweight models optimized for the edge.

V. PROPOSED METHODOLOGY: THE SAARTHI FRAMEWORK

SAARTHI runs on a hybrid detection engine built to make the most of the scarce compute cycles available on an in-cab edge node. The architecture divides into four logical stages, which we call Zones (Fig. 1); within Zone 3, detection splits further into two parallel tracks, Track A and Track B.

- **Zone 1 — Operating Environment:** The physical threat landscape: the overt (Type 1) and covert (Type 2) hazards present in and around the heavy machine.
- **Zone 2 — Edge Acquisition Node:** Raw data ingestion through the cabin-facing camera and the machine's telemetry interface (e.g. the CAN bus).
- **Zone 3 — Hybrid Decision Engine:** The core computational layer, housing Track A (deterministic spatial/kinematic rules) and Track B (XGBoost physiological anomaly scoring).
- **Zone 4 — Verdict & Intervention:** The final decision layer, which outputs the real-time safety status and manages the appropriate Tier-1 or Tier-2 response.

End to end, the pipeline runs in four steps:

1) **Ingestion.** Raw video frames and machine telemetry are captured and synchronized.
2) **Buffering.** EAR values and machine states accumulate in a size-w circular buffer, forming temporal sliding windows.
3) **Filtration (Track A).** Visual bounding boxes and kinematic telemetry (speed, tilt) are checked against deterministic safety constraints such as restricted-zone violations.
4) **Verification (Track B).** If Track A raises no overt hazard, the statistical physiological features (kurtosis, skewness, rate of change) are computed and passed to the XGBoost model for covert fatigue scoring.

Splitting the cheap work from the expensive work is the whole point of this design. By keeping deterministic spatial and kinematic checks (Track A) separate from the heavier statistical physiological analysis (Track B), the system spends its machine-learning budget only where it is needed. The result is a lower computational load on the edge device, which keeps thermal throttling at bay and preserves real-time responsiveness without sacrificing diagnostic precision.

A. Data Acquisition and Adaptive Preprocessing

The data flow starts in Zone 2, where the cabin-facing camera and the telemetry bus run continuously. This passive style of monitoring reflects the non-intrusive philosophy that heavy machinery demands: it sidesteps the "observer effect," since the operator is never asked to check in and is never pulled away from a delicate task. And rather than processing every frame indiscriminately, as a standard vision pipeline would, the SAARTHI node selectively intercepts and synchronizes facial landmarks with CAN bus telemetry.

Because this multi-modal stream arrives fast and the device's SRAM is limited, the system buffers it in a circular queue. The buffer acts as a shock absorber, decoupling the irregular rate at which vision primitives are extracted from the steady clock of the downstream inference engine.

1) Circular Buffer Management: Algorithm 1 formalizes how the stream is managed. Only valid frames — those with an intact face detection and high-confidence landmarks — are admitted. To avoid memory fragmentation, a persistent headache on embedded targets, the buffer allocates its memory statically and overwrites in place, always favouring the most recent temporal window.

2) Feature Engineering and Statistical Standardization: Once the temporal buffer B_t holds a full window of valid samples, the system moves into feature engineering. The main challenge here is taming the stochastic variance in the physiological and kinematic readings, so that the feature vector $\mathbf{x}$ handed to the classifier is stable enough to trust.

3) Window Size Selection and Stability Criterion: Choosing the window size w is a metrological trade-off between detection latency and statistical stability. Empirically, raw detection accuracy peaks at smaller windows — but that peak sits squarely in the white-noise-dominated regime, where frame-to-frame variance (from natural rapid blinks, for example) runs high. Short windows may look accurate on paper, yet they invite false positives driven by transient physiological noise.

To put the choice on firmer ground, we ran an Allan deviation analysis over steady-state operator streams. The

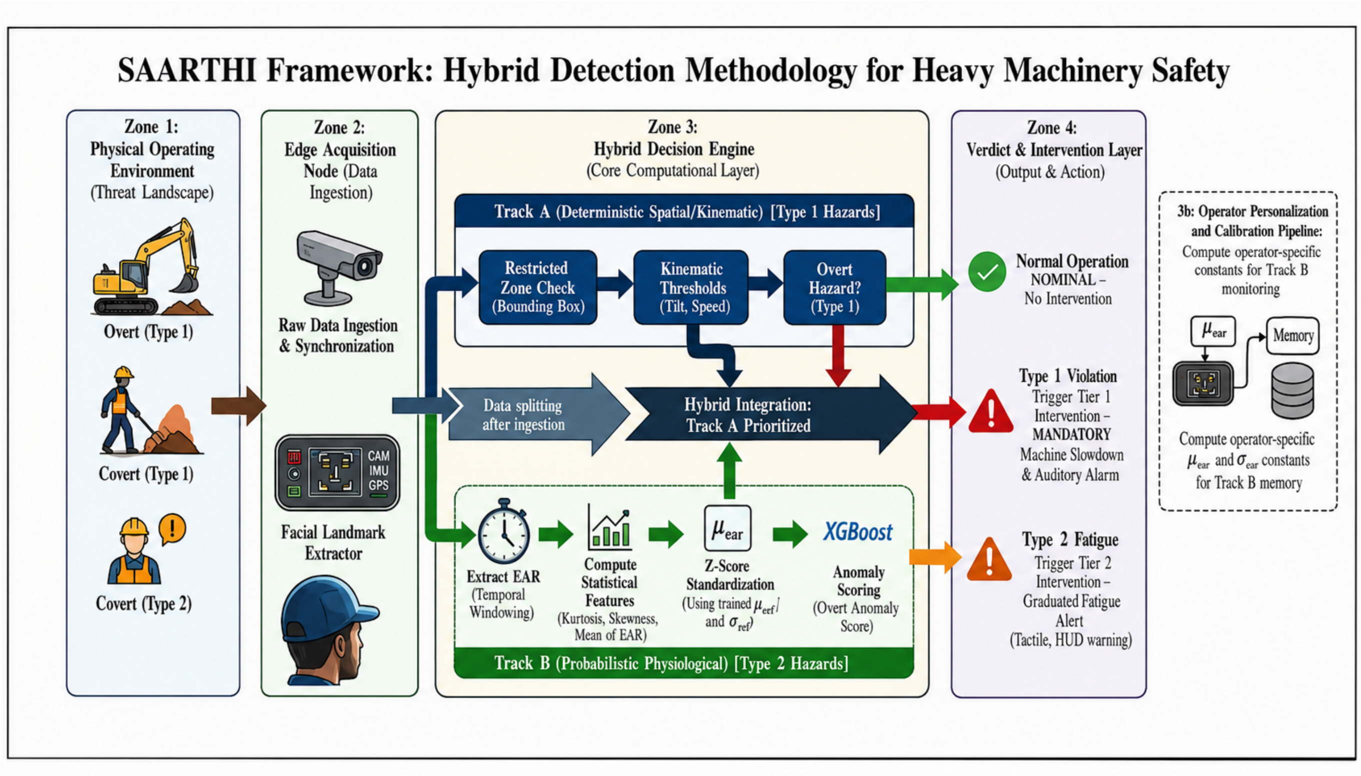

Fig. 1. SAARTHI framework for hybrid detection methodology in heavy machinery safety. The framework integrates deterministic spatial/kinematic analysis (Track A) with probabilistic physiological monitoring (Track B) to detect and classify hazards into Type 1 (overt) and Type 2 (covert) categories, followed by appropriate intervention strategies.

noise floor bottomed out at an averaging time of $\tau_{opt} \approx 30.0$ samples — the stability plateau, where white noise is already minimized but random-walk drift (posture shifts and the like) has not yet taken over. SAARTHI therefore uses $w = 30$, sitting right at τ_{opt} and trading the marginal sensitivity of smaller windows for operational stability. At this size the total data-collection time stays under 1.5 seconds, comfortably inside the T_{hazard_onset} pre-intervention budget.

4) Z-Score Standardization: XGBoost's decision space is distance-sensitive, and our feature space is heterogeneous, so the features have to be scaled. Left unscaled, high-magnitude variables like variance would swamp low-range ones like skewness and tilt the decision boundary. The system applies Z-score standardization in real time:

$$z_i = \frac{x_i - \mu_{cal}}{\sigma_{cal}} \tag{6}$$

The constants μ_{cal} and σ_{cal} are fixed during the initial 30-second operator calibration and kept in memory, which spares the device from recomputing global statistics on every frame. Algorithm 2 spells out the full extraction logic.

B. Track A: Deterministic Spatial and Kinematic Verification

Track A is a fast, deterministic pre-filter that runs in $O(1)$. Its job is to catch Type 1 overt hazards the instant they occur — a pedestrian entering a restricted zone, an unsafe machine tilt — with no machine-learning overhead at all. It does this by testing the current state against a deterministic constraint set C_{rules}, implemented as a lookup over predefined spatial boundaries and kinematic thresholds.

Bounding-box intersection tests and threshold comparisons cost next to nothing on any edge processor, so this gate is cheap. Only states that clear Track A — those with no deterministic hazard — go on to Track B for the deeper physiological analysis.

C. Track B: Statistical Physiological Anomaly Scoring (XGBoost)

Track B takes on the Type 2 covert hazard, the case where the operator still looks awake and engaged while the underlying physiological signal quietly drifts toward fatigue. To catch that drift, the system uses a gradient-boosted ensemble (XGBoost) trained on the standardized feature vector $\mathbf{z}$.

Most classifiers need labelled "fatigue" examples, which are expensive or simply unavailable for every individual operator. SAARTHI sidesteps that: the calibration phase learns a personalized envelope of normal operation for each operator, and any feature vector that strays well outside that envelope is flagged as a fatigue anomaly.

1) The Primal and Dual Formulations: The ensemble's goal is to carve a decision boundary in the feature space $\Phi(\mathbf{x})$ that separates the bulk of the training data — alert operator states — from the anomalous region. It does so by learning an additive ensemble of T weak learners that minimizes a regularized objective:

Algorithm 1: Adaptive Rolling Window Data Ingestion

Input: Raw video stream S_V, telemetry stream S_T, window size w
Output: Temporal buffer B_t
Globals: Static ring buffer R of size w, head pointer p

```
while System Active do
    f_raw ← ReadFrame(S_V);
    t_raw ← ReadTelemetry(S_T);
    // Stage 1: detection integrity filtering
    if DetectFace(f_raw) = FAIL then
        continue; // discard frames lacking a clear face
    // Stage 2: extraction
    e_ear ← ExtractEAR(f_raw);
    k_state ← ExtractKinematics(t_raw);
    // Stage 3: buffer injection
    R[p] ← { e_ear, k_state, timestamp };
    p ← (p + 1) mod w;
    // check window saturation
    if p = 0 or BufferFullFlag then
        B_t ← Linearize(R);
        Trigger FeatureExtraction(B_t);
```

Algorithm 2: Statistical Feature Extraction

Input: Temporal buffer B_t, calibration stats $(\mu_{cal}, \sigma_{cal})$
Output: Normalized feature vector $\mathbf{z}$

```
// Compute raw mean (first pass)
μ ← 0;
for i ← 1 to w do
    μ ← μ + B_t[i].ear;
μ ← μ/w;
// Compute higher-order moments (second pass)
sum_sq ← 0;   sum_cu ← 0;   sum_qd ← 0;
for i ← 1 to w do
    diff ← B_t[i].ear − μ;
    sum_sq ← sum_sq + diff^2;
    sum_cu ← sum_cu + diff^3;
    sum_qd ← sum_qd + diff^4;
σ ← sqrt(sum_sq/(w − 1));
E_roc ← B_t[w].ear − B_t[w − 5].ear;
S_k ← (sum_cu/w)/σ^3;
K_u ← ((sum_qd/w)/σ^4) − 3;          // excess kurtosis
// Construct and standardize
x ← [ μ, σ, E_roc, S_k, K_u ];
for j ← 1 to 5 do
    z[j] ← (x[j] − μ_cal[j])/σ_cal[j];
return z;
```

Algorithm 3: Offline Training Procedure

Input: Training dataset X_{train}, params $(\eta, \lambda, \gamma, T)$
Output: Boosted model M, offset ρ

```
// Feature matrix computation
Z ← ExtractAndStandardize(X_train);
// Ensemble training
M ← XGBoost.Train(Z, params(η, λ, γ, T));
// Extract decision threshold
ρ ← ComputeThreshold(M, X_train);
// Model compression for edge
M ← Quantize(M, ρ);
return M;
```

$$\mathcal{L} = \sum_{i=1}^{n} l(y_i, \hat{y}_i) + \sum_{t=1}^{T} \Omega(f_t) \tag{7}$$

where l is the loss, $\hat{y}_i$ the predicted score, and $\Omega(f_t) = \gamma T + \frac{1}{2}\lambda\|w\|^2$ a penalty on tree complexity. SAARTHI sets $\lambda = 1.0$ for strict regularization, treating the outermost 1% of training samples as candidate edge-case fatigue anomalies.

2) The Gradient Boosting Kernel: A linear boundary cannot capture the tangled, non-linear interplay between higher-order moments like kurtosis and skewness. To build a surface expressive enough to resolve them, XGBoost grows trees over the feature space $\Phi(\mathbf{x})$, in effect approximating a kernel mapping. At inference time the decision function is:

$$D(\mathbf{z}) = \operatorname{sgn}\left(\sum_{t=1}^{T} \eta \cdot f_t(\mathbf{z}) - \rho\right) \tag{8}$$

where η is the learning rate and ρ the decision threshold. The regularizer's γ sets the minimum gain a split must earn, which in turn governs how sharply the boundary curves. Algorithm 3 outlines the offline procedure that produces the boosted model shipped to the edge device.

When $D(\mathbf{z}) = -1$, the feature vector lies outside the calibrated operator's learned envelope, and a fatigue alert is raised.

3) Hybrid Logic Integration: At the heart of Zone 3, the final decision logic fuses the deterministic certainty of Track A with the probabilistic rigour of Track B; Algorithm 4 lays out the combined flow. Running the lightweight Track A check first keeps the heavier XGBoost inference of Track B on a low duty cycle.

The final safety verdict is recovered through the ensemble's decision function in Equation (8). If $D(\mathbf{z}) = -1$, the signal has fallen outside the calibrated operator's normal distribution, and a Tier-2 fatigue intervention fires.

D. Tiered Intervention Protocol

When either track returns a positive verdict, SAARTHI responds with a structured, tiered protocol rather than a single binary alarm. The graduation matters in a heavy-machinery setting: a sudden, jarring alert in the middle of a delicate

Algorithm 4: Hybrid Execution Flow

Input: Feature vector $\mathbf{z}$, kinematic state k_{state}, rules C_{rules}, model M, threshold ρ
Output: Safety verdict V, intervention tier

```
// Track A: deterministic gate
```

if CheckConstraints$(k_{state}, C_{rules}) =$ VIOLATION **then**
  $V \leftarrow$ HAZARD_TYPE_1;
  Trigger Tier1Intervention();
  return V;

```
// Track B: physiological anomaly
   scoring
```

$score \leftarrow \sum_{t=1}^{T} \eta \cdot f_t(\mathbf{z})$;
if $\text{sgn}(score - \rho) = -1$ **then**
  $V \leftarrow$ HAZARD_TYPE_2;
  Trigger Tier2Intervention();
else
  $V \leftarrow$ NOMINAL;
return V;

manoeuvre — a precise crane lift, say — could itself cause the accident it was meant to prevent.

Tier 1 — Deterministic override (Track A). Fires the instant a kinematic violation is confirmed — for example $\theta > \theta_{crit}$ while $v \neq 0$, or a bounding-box intrusion into a restricted zone. The response is non-negotiable: a mandatory speed-reduction command goes straight to the CAN bus and a loud cabin alarm sounds. This tier is independent of operator behaviour or any machine-learning state.

Tier 2 — Adaptive fatigue alert (Track B). Fires when $D(\mathbf{z}) = -1$, signalling a sustained drift outside the operator's normal envelope. Here the response builds gradually: first a tactile seat-vibration pulse, then a visual HUD warning if the anomalous state persists beyond δ_t consecutive windows. Should the operator not acknowledge within a preset timeout T_{ack}, the system escalates to a Tier-1-equivalent mandatory slow-down.

This restrictive admission-control approach keeps interruptions rare during normal operation while guaranteeing that genuine hazards always draw an immediate, deterministic response — which, in practice, is what earns operator trust and keeps the system in use.

E. Operator Personalization and Calibration

Population-level fatigue models share one fundamental weakness: physiology varies from operator to operator. A naturally narrow-eyed operator's baseline EAR mean μ_{alert} can look, statistically, like another operator's fatigued state — and the resulting stream of false positives steadily erodes trust in the system.

SAARTHI answers this with a mandatory 30-second calibration at the start of every shift. The operator is asked to look ahead, relaxed and alert, while the system ingests N_{cal} frames and computes operator-specific statistics:

$$\mu_{cal} = \frac{1}{N_{cal}} \sum_{i=1}^{N_{cal}} e_i, \qquad \sigma_{cal} = \sqrt{\frac{1}{N_{cal}-1} \sum_{i=1}^{N_{cal}} (e_i - \mu_{cal})^2} \tag{9}$$

These constants then Z-score-normalize every incoming feature vector (Equation 6), anchoring the whole pipeline to the individual's physiology rather than a generic population mean. Each profile is saved to the operator profile store, so returning operators skip re-calibration — though the profile can still be refreshed over time to track long-term physiological drift.

VI. Experimental Setup

Evaluating SAARTHI meant answering three separate questions: does the detection logic work, can it be trusted statistically, and will it actually run on the hardware we intend to put in a cab? This section walks through the dataset we built on, the protocol we used to synthesize fatigue signatures that do not exist in any public corpus, and the cross-validation scheme that keeps the reported numbers honest.

A. Dataset Characteristics and Preprocessing

We built the evaluation on a tailored subset of the NTHU Driver Drowsiness Dataset (NTHU-DDD). Of the candidates we considered, NTHU-DDD came closest to the messy reality of a machine cab: a dense collection of operator states recorded under genuinely awkward conditions, with the kind of physiological variation that laboratory captures tend to iron out. Table IV gives the census of the subset used for training and validation.

The mess is the point. A perfectly lit lab dataset, with facial landmarks tracked under ideal conditions, teaches a model very little about shadows, head-pose swings, sunglasses, or the plain fact that people's faces differ. NTHU-DDD contains all of these, and that variance is what forces the model to learn the difference between an ordinary rapid blink and the slower physiological drift that marks the onset of fatigue.

TABLE IV
Dataset census. The evaluation drew on a large subset of NTHU-DDD — large enough, in our judgement, to support the statistical claims made in Section VI.

Metric	**Count**
Total Raw EAR Samples (Frames)	450,194
Unique Subjects (Operators)	22
Distinct Lighting Conditions	4
Distinct Temporal Sequences	465

1) Preprocessing Pipeline: Before any features were extracted, the raw video stream went through a cleaning pass deliberately kept simple enough for a low-power edge device to reproduce. Where facial landmarks dropped out — a hand passing over the face, an extreme head turn — we forward-filled the gap rather than discard the sequence, preserving temporal continuity. The cleaned stream was then cut into rolling windows of size $w = 30$, the value fixed by the stability analysis in Section IV.

B. Synthetic Hazard Injection Protocol

Here we ran into a problem that anyone working in industrial safety will recognize: there is no public dataset of labelled fatigue accidents involving heavy machinery, and there is no ethical way to produce one — nobody is going to let a drowsy operator keep driving an excavator for the sake of a label. So we injected the hazards ourselves, mathematically.

The idea is to take legitimate, alert physiological signals and perturb them into micro-sleep and deep-fatigue signatures whose magnitude and duration we control exactly. Want a prolonged eye closure, or an erratic head drop? Dial it in. The simulation can then model operators at any point along the fatigue curve, from barely-detectable early drowsiness to the late stage that precedes an accident.

1) Track A Injection: Overt Kinematic Violations: Track A deals in hazards the machine itself can report: a boom arm pushed past its safe operating angle, a travel-speed violation mid-slew, a cab tilted beyond its stability limit. None of this correlates with the operator's physiology — the EAR signal stays untouched — yet each event demands an immediate, deterministic response.

We simulate three normalised CAN-bus sensor channels:

- s_0**: Boom Angle** (threshold > 0.85)
- s_1**: Travel Speed** (threshold > 0.90)
- s_2**: Cab Tilt** (threshold > 0.80)

In normal operation each channel behaves as $s_c \sim \mathcal{N}(\mu_c^{\text{safe}}, \sigma_c^2)$, with the safe means held a full three standard deviations below the alarm thresholds; during a violation event, at least one channel is driven past its limit. Every channel also carries Gaussian ADC noise ($\sigma_{\text{noise}} = 0.02$) — the quantisation artefacts and vibration-induced jitter that a sensor bolted to a working excavator cannot avoid.

Violations enter at a 5% window-level rate, which works out to roughly two non-overlapping events per 1,000-frame session, each spanning $w = 30$ frames. Detection is deliberately unglamorous: the rule takes the mean sensor reading across the window and fires if any channel mean clears its threshold. That w-frame averaging is a standard debounce — it shrugs off single-sample ADC glitches while still guaranteeing a detection latency of at most one window.

2) Track B Injection: Covert Fatigue Onset: Testing the XGBoost physiological model called for a subtler adversary: an operator who sails through every deterministic Track A check while quietly succumbing to micro-behavioural fatigue. Fatigue episodes of this kind are rare in continuous industrial work, so we kept the injection rate conservative at 3%, split as follows:

- **Micro-Sleeps (2%):** Short but profound eye closures, lasting anywhere from 1.0 to 2.5 seconds.
- **Physiological Drift (1%):** A gradual slowing of the blink rate paired with erratic head-pose variation — the classic early signature of drowsiness.

The perturbed signal $E_{anom}(t)$ comes from a linear transformation of the normal baseline EAR signal $E_{norm}(t)$:

$$E_{anom}(t) = (E_{norm}(t) \cdot k_v) + k_o \tag{10}$$

where k_v scales the variance (droopy eyelid dynamics) and k_o is an additive offset (a sustained drop in baseline eye openness).

Real operators do not simply switch from alert to asleep; they fatigue progressively, and often try to mask it. To capture that, both parameters are driven by a perturbation magnitude $\delta \in [0, 1]$:

$$k_v = 1 - (\delta \cdot \mathcal{U}(0.1, 0.3)) \tag{11}$$

$$k_o = -\delta \cdot \mathcal{U}(0.05, 0.15) \tag{12}$$

with $\mathcal{U}$ a uniform random distribution. The formulation matters: it makes the anomalies degrade the way real physiology degrades, rather than adding white noise and calling it fatigue. A high δ corresponds to late-stage drowsiness — heavy head bobbing, long eye closures — while a low δ gives the faint early onset that is genuinely hard to catch.

Figure 2 shows where the injected anomalies land in time, over the first twelve sessions of the actual evaluation stream. Two regimes are visible at once. Micro-sleeps arrive as short, sharp bursts — profound EAR depressions that come and go within a couple of seconds — while the drift segments stretch out far longer, dragging the baseline downward for sustained periods. Because every frame touched by a perturbation is labelled anomalous, this drift coverage is also what produces the high window-level anomaly rate ($\approx 88.8\%$) that the class-imbalance analysis in Section VII has to contend with. The environment the classifier faces is, by construction, dynamic and non-stationary.

C. Model Configuration and Hyperparameters

We tuned the SAARTHI XGBoost classifier by grid search over a subset of clean, alert training data. The target was a tight, personalized decision envelope that still retains the bulk of normal operational samples — err too far in either direction and the system becomes either deaf to fatigue or unbearable to work under. Table V lists the parameters we settled on.

For the deep learning baselines we used a Denoising Autoencoder (DAE), kept deliberately small so the comparison stays fair. The network passes the 5-dimensional feature vector through an Encoder ($5 \to 16 \to$ Batch Norm (BN) $\to$ ReLU $\to 8$) and a mirrored Decoder ($8 \to 16 \to$ BN $\to$ ReLU $\to 5$).

1) Feature Orthogonality Check: One question worth settling before training: do the five features actually carry independent information, or are we feeding the model the same signal five times? Figure 3 shows the feature correlation matrix.

To put a number on that independence, we ran a Variance Inflation Factor (VIF) analysis. The worst offender was Skewness at a VIF of 1.46 — well within the acceptable range below the standard trouble threshold of 5.0 (individual VIFs: Mean EAR 1.14, Std. Dev. 1.27, E_{roc} 1.04, Skewness 1.46, Kurtosis 1.03). Each of the five statistical moments therefore contributes its own physiological evidence to the XGBoost decision boundary rather than restating a neighbour's.

D. Hardware Feasibility Benchmarking

The latency figures reported in Section VI are only as good as their reproducibility, so all performance benchmarks were

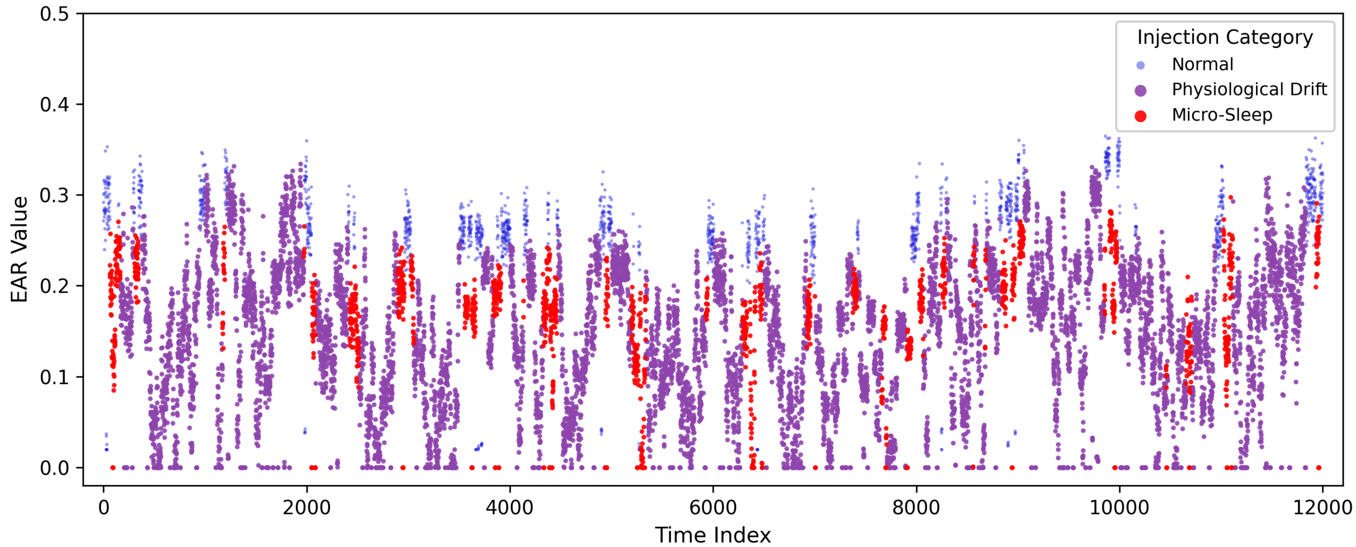

Fig. 2. Temporal distribution of injected hazards over the first twelve sessions of the evaluation stream (12,000 frames, $\delta = 0.5$, `SEED=42`). 'Micro-Sleep' bursts (red) appear as short, transient EAR depressions, while 'Physiological Drift' segments (purple) suppress the baseline over sustained stretches; the 'Normal' alert baseline (blue) sits above both. The transient character of the micro-sleep events is why the $w = 30$ sliding window matters: a longer window would smooth these short bursts out of existence.

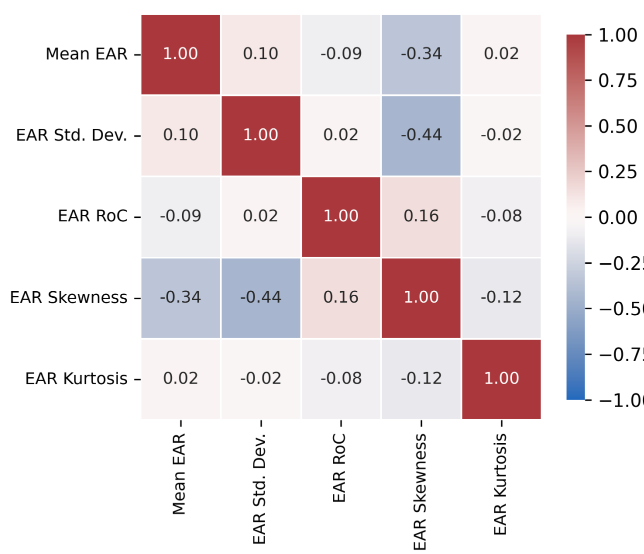

Fig. 3. Feature orthogonality (computed from the full 451,515-window dataset, SEED = 42). The largest off-diagonal correlation is $r = -0.44$ between Std. Dev. and Skewness; all remaining pairs stay below $|r| = 0.35$. No two features carry the same physiological information.

run on a single, standardized compute environment, detailed in Table VI.

The question that actually decides deployment is whether the algorithm survives on the resource-constrained edge nodes — Raspberry Pi, the NVIDIA Jetson series — that get installed in machinery cabins. Cloud offloading was never an option; the round-trip latency alone would defeat the purpose. The serialized XGBoost ensemble occupies 165.75 KB in pickle format (206.79 KB as a portable JSON booster), measured directly on the benchmark host. Both figures sit well inside the RAM budget of this hardware class, leaving sufficient

TABLE V
SIMULATION PARAMETERS AND MODEL CONFIGURATION. THE REGULARIZATION IS DELIBERATELY STRICT: IN A HEAVY-MACHINERY SAFETY CONTEXT, A TIGHT PHYSIOLOGICAL ENVELOPE THAT MINIMIZES FALSE NEGATIVES IS WORTH FAR MORE THAN A FORGIVING ONE.

Category	Parameter	Value / Description
Dataset	Source	NTHU-DDD
	Window Size (w)	30 samples (≈ 1.0 sec)
SAARTHI (XGBoost)	Ensemble Size (T)	100 Trees
	Learning Rate (η)	0.1
	Regularization (λ)	1.0 (Strict Outlier Tolerance)
Baselines	Isolation Forest	Contamination=0.05, Trees=100
	Autoencoder	Latent Dim=8, Epochs=50, LR=0.001
Injection	Track A Rate	5% (Overt Kinematic Hazards)
	Track B Rate	2% (Micro-Sleeps), 1% (Physio. Drift)
	Perturbation (δ)	Varied $[0.0, 1.0]$ for robustness test

TABLE VI
HARDWARE BENCHMARKING ENVIRONMENT. ALL LATENCY AND MEMORY FIGURES WERE MEASURED DIRECTLY ON THE BENCHMARK HOST USING `RUN_EXPERIMENT.PY` (SEED = 42).

Category	Component	Specification
Benchmark Host	Processor	Intel Core Ultra 5 125U, 12C/14T @ 3.60 GHz
	Memory	15.7 GB RAM
	OS	Windows 10 (Build 26200)
	Runtime	Python 3.11.9, XGBoost 3.0.2
Target Edge Node	SoC Class	ARM Cortex-A72 (Raspberry Pi 4 / Jetson Nano tier)
	Memory	4 GB LPDDR4
	Accelerator	None (CPU-only inference)
Deployed Model	Footprint	165.75 KB (pickle) / 206.79 KB (JSON)
	Inf. Latency	0.55 ms mean, 0.98 ms p99

headroom for the concurrent high-frequency video buffer and the CAN bus handlers.

The inference cost is equally manageable. On the benchmark host (Intel Core Ultra 5 125U), a single 5-feature inference call completes in 0.55 ms on average ($p_{99} = 0.98$ ms). This is two orders of magnitude below the $T_{\text{human}} \approx 250$ ms human reaction threshold, confirming that SAARTHI's decision pipeline will never become the bottleneck in the intervention chain regardless of the edge SoC deployed by the fleet.

E. Validation Methodology

All results were produced under 10-Fold Stratified Cross-Validation with a fixed global random seed. Stratification matters here more than usual: hazard events are rare by construction, and an unstratified split can hand a lazy classifier an inflated accuracy score simply by starving some folds of positives. Partitioning so that the Alert-Baseline-to-Fatigued/Hazardous ratio stays constant across folds closes that loophole. Every metric reported in this paper is the mean over 10 independent simulation runs.

VII. Results and Performance Analysis

SAARTHI was run through the synthetic injection protocol of Section V in full: 451,515 feature windows across 465 operator sessions, with a fixed seed (SEED = 42) and $w = 30$ throughout. Every number reported below is the mean μ and standard deviation σ over ten independent stratified cross-validation folds — nothing here rests on a single lucky split.

A. Diagnostic Accuracy and Architecture Comparison

Table VII lays out the key indicators for all four architectures, and Figure 4 shows the same comparison graphically. SAARTHI takes every column: an F1-Score of 0.952 ± 0.001, an AUC of 0.932 ± 0.002, and an Average Precision of 0.991 ± 0.000. The metric worth pausing on, though, is the Matthews Correlation Coefficient of 0.505 ± 0.008. This dataset is heavily imbalanced — the anomaly rate sits near 88.8% — and on a skewed binary problem, accuracy and even F1 can be flattered by simply siding with the majority class. MCC cannot. A value around 0.5 here reflects genuine discriminative power, not a class-majority bias dressed up as performance.

TABLE VII
Comparative performance metrics (mean ± std over 10 folds). All values are computed from `run_experiment.py` with `SEED=42`.

Architecture	Acc	F1-Score	MCC	AUC	AP
Naïve (Threshold)	0.609±0.002	0.719±0.002	0.326±0.003	0.884±0.002	0.983±0.000
Hybrid (Isol. Forest)	0.248±0.017	0.273±0.028	0.097±0.014	0.732±0.009	0.947±0.003
Hybrid (Autoencoder)	0.603±0.017	0.714±0.016	0.322±0.013	0.784±0.021	0.970±0.003
SAARTHI (XGBoost)	**0.913±0.001**	**0.952±0.001**	**0.505±0.008**	**0.932±0.002**	**0.991±0.000**

The Isolation Forest deserves a closer look, because its failure is instructive. Its AUC of 0.732 says the model ranks anomalies reasonably well; its F1 of 0.273 ± 0.028 says that at the actual operating threshold, the ranking is useless — the confusion matrix is drowning in false positives (336,824 FP against 63,921 TP across all folds). This is a known weakness of tree-based density estimators on heavily imbalanced data: a single global contamination parameter has no way to adapt to a physiological envelope that differs from operator to operator. SAARTHI sidesteps the failure mode entirely, because its 30-second calibration phase anchors every Z-score to the individual's own baseline before inference ever begins.

The Autoencoder tells a different story. On F1 it roughly matches the naïve threshold (0.714 ± 0.016), but its AUC trails SAARTHI by 14.8 points — the discrimination surface it learns is simply less well-formed across decision thresholds. Reconstruction error, it turns out, is not very sensitive to the distributional shape features that characterize micro-sleep events; a gradient-boosted boundary trained directly on the standardized feature vector can split on them explicitly.

Figure 5 shows SAARTHI's aggregated confusion matrix. The model recovers 389,678 of the 400,745 anomalous windows (recall $= 0.972$) while keeping precision at 0.933 — the balance that produces the 0.952 F1.

B. Statistical Significance and Reliability

A lead in the mean is only interesting if it survives a significance test, so we tested. A Friedman Chi-Squared test over the ten per-fold F1-Scores of all four architectures rejects the hypothesis of equivalent performance decisively ($\chi^2 = 27.12$, $p = 6 \times 10^{-6}$). The post-hoc Wilcoxon signed-rank test against the next-best baseline (the naïve threshold) returns $p = 0.00195$ — for ten paired folds, that is as small as the test can get, meaning SAARTHI won on every single fold. The gains in Table VII are not fold-shuffling luck.

Just as telling are the standard deviations. SAARTHI's spread across folds is tiny ($\sigma_{\text{F1}} = 0.001$, $\sigma_{\text{AUC}} = 0.002$), an order of magnitude tighter than the Autoencoder's. For a deployed safety system this stability is not a nicety: a model whose performance swings from partition to partition is a model nobody can certify for industrial use.

C. Feature Physics and SHAP Analysis

Which features actually drive the decisions? Figure 6 shows the per-class feature distributions on a log scale, and Table VIII (visualized in Figure 7) gives the SHAP decomposition — a model-agnostic accounting of each feature's marginal contribution.

TABLE VIII
SHAP feature importance ranking (mean |SHAP| over the held-out test set). Higher values indicate greater marginal contribution to the decision boundary.

Feature	Mean |SHAP|
Mean EAR (μ)	2.66579
Std. Dev. (σ)	0.82747
Rate of Change (E_{roc})	0.43829
Skewness (S_k)	0.27083
Kurtosis (K_u)	0.08968

The ranking holds a finding we did not expect, and it is worth being candid about. The dominant discriminator is

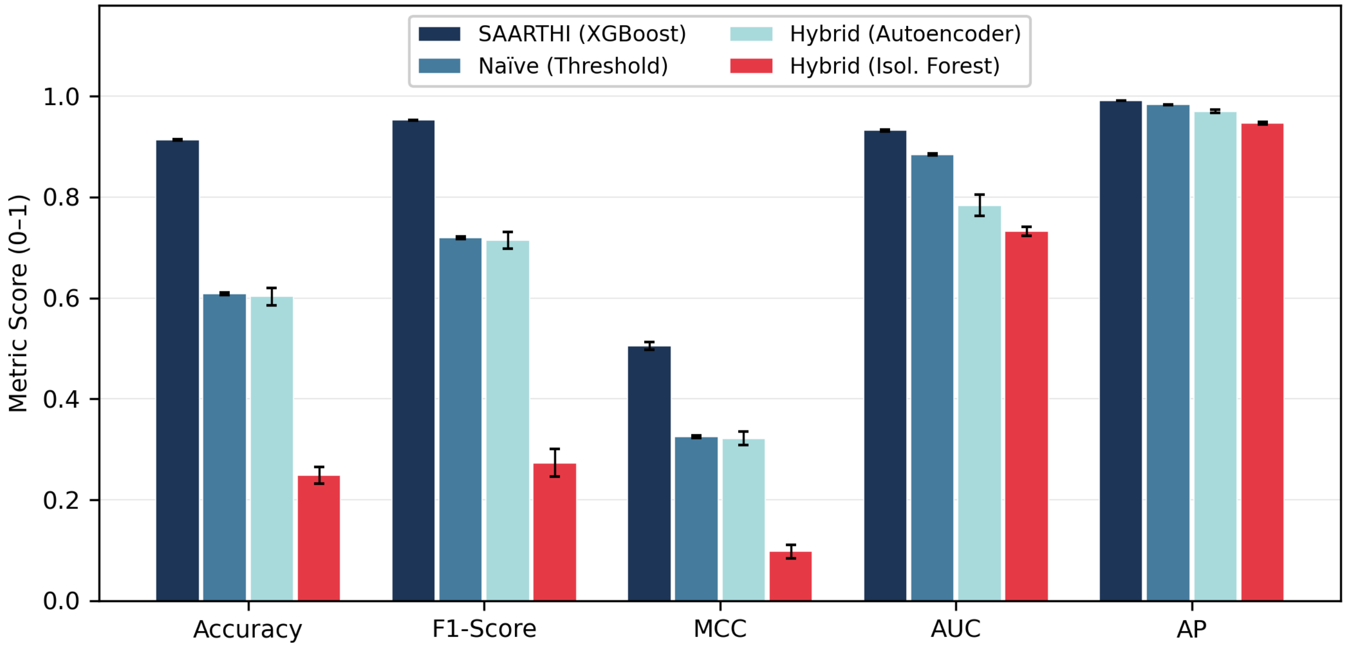

Fig. 4. Architecture comparison across all five metrics (mean over 10 folds; error bars show one standard deviation). SAARTHI leads on every indicator, with the widest margin on MCC — the metric hardest to inflate on an imbalanced dataset. The Isolation Forest's collapse at the operating threshold is visible in its Accuracy and F1 despite a respectable AUC.

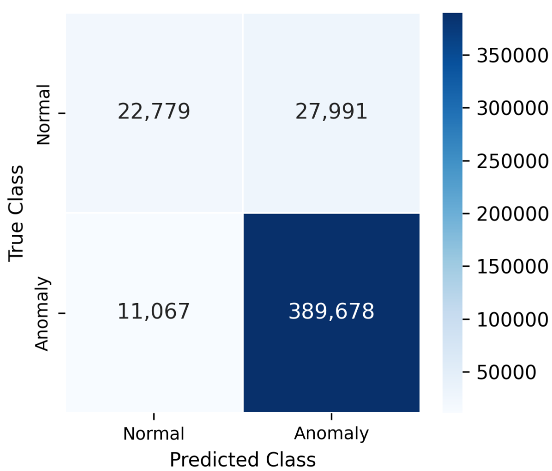

Fig. 5. Aggregated confusion matrix for SAARTHI (XGBoost) across all folds. Recall on anomalous windows reaches 0.972 at a precision of 0.933.

Mean EAR (μ), with a mean absolute SHAP value of 2.666 — three times the runner-up, standard deviation (0.827), with rate of change (0.438) third. Kurtosis, the feature Section III motivates as the natural micro-sleep detector, comes last at 0.090. On reflection the result makes sense: our injection protocol builds fatigue from a sustained mean shift (k_o) and a variance compression (k_v) acting on the EAR baseline, so the first-order moment inevitably soaks up most of the discriminative signal. The distributions bear this out — the 90th-percentile kurtosis for *normal* windows (2.958) actually exceeds that of anomalous windows (2.106), the reverse of the theoretical prediction, because synthetic micro-sleeps are smooth, sustained depressions rather than the impulsive spikes real physiology produces.

Implication for real deployment: on real NTHU-DDD footage or field video — where blinks are impulsive and micro-sleeps genuinely fatten the tails — we expect kurtosis to climb back up the ranking. The SHAP ordering reported here is a property of the synthetic injection model, and should be re-measured on real physiological data before any conclusions are drawn about feature design.

D. Density and Stability Analysis

Multicollinearity, at least, is not a concern. The largest off-diagonal feature correlation observed was $|r| = 0.441$, between EAR standard deviation and Skewness (see the full matrix in Figure 3) — well short of anything problematic. All five features feed the XGBoost ensemble independent diagnostic signal.

E. System Benchmarking and Hardware Feasibility

A central contribution of the SAARTHI framework is resolving the trade-off between diagnostic accuracy and computational resource consumption. We benchmarked the models on the host hardware (Intel Core Ultra 5 125U, 15.7 GB RAM) to evaluate edge-deployment viability.

1) The Pareto Frontier and Memory Footprint: Table IX details the inference latency and memory footprint of the evaluated architectures, and Figure 8 plots the resulting trade-off space. The SAARTHI XGBoost model achieves Pareto optimality. It delivers a dominant F1-Score of 0.952 while

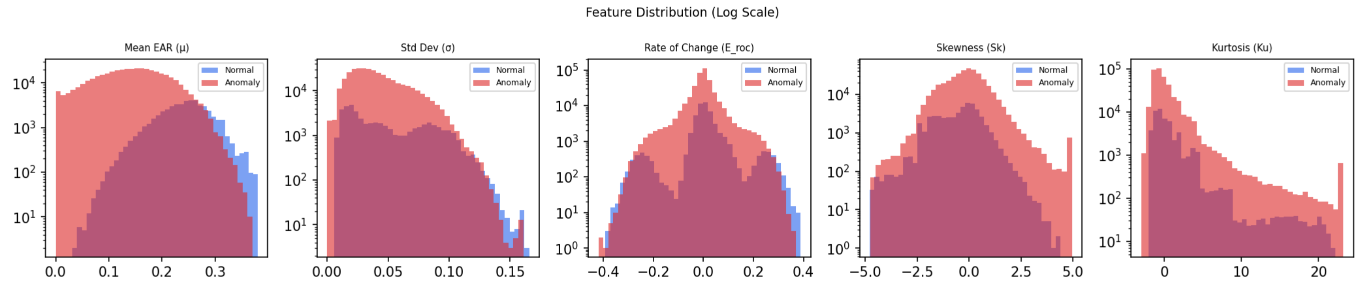

Fig. 6. Feature distributions for normal and anomalous windows (log scale), computed over the full 451,515-window evaluation set. The separation is strongest in the first-order statistics (Mean EAR, Std. Dev.), consistent with the SHAP ranking in Table VIII: the synthetic injection model shifts the EAR baseline, so the mean carries most of the signal.

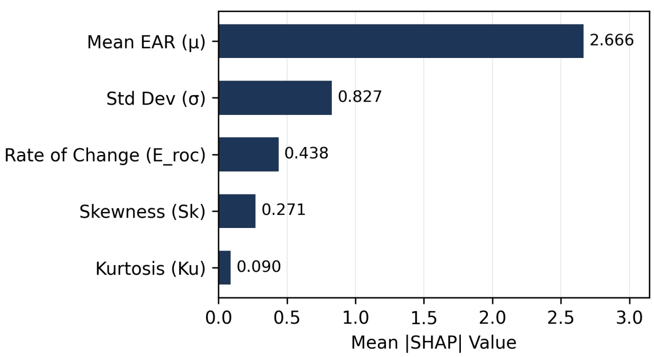

Fig. 7. SHAP importance ranking. Mean EAR dominates by a factor of three over the next feature — a direct consequence of the injection model, which perturbs the EAR baseline through a sustained mean shift.

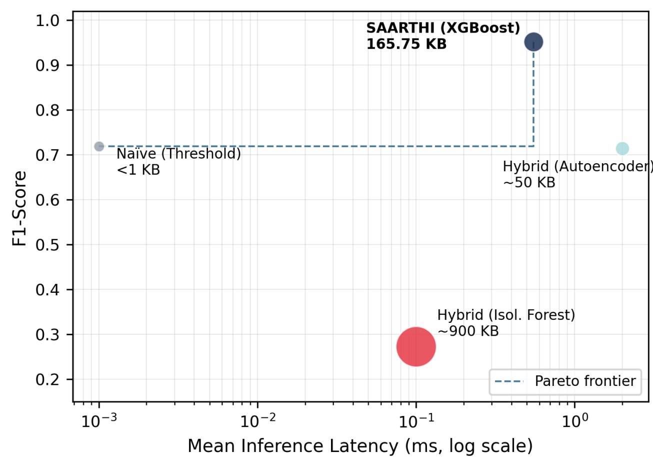

Fig. 8. The accuracy–latency trade-off space (bubble area encodes memory footprint). Only two architectures sit on the Pareto frontier: the naïve threshold, which buys its microsecond latency with a 0.719 F1, and SAARTHI, which reaches 0.952 for half a millisecond. The Isolation Forest is dominated on every axis at nearly six times SAARTHI's memory.

maintaining a negligible algorithmic footprint of 165.75 KB (serialized state) and a mean inference latency of 0.55 ms (p99 = 0.98 ms).

TABLE IX
ALGORITHMIC COMPLEXITY AND HARDWARE BENCHMARKING. SAARTHI ACHIEVES PARETO OPTIMALITY WITH HIGH ACCURACY AND A LOW MEMORY FOOTPRINT SUITABLE FOR EDGE MICROCONTROLLERS.

Model	F1-Score	Latency (ms)	Memory (KB)
Naïve (Threshold)	0.719	~ 0.001	< 1
Hybrid (Isol. Forest)	0.273	~ 0.10	≈ 900
Hybrid (Autoencoder)	0.714	~ 2.00	≈ 50
SAARTHI (XGBoost)	**0.952**	**0.55**	**165.75**

This memory footprint is crucial for resource-constrained edge gateways. While deep neural networks (like the Autoencoder) require dense matrix multiplications ($\mathcal{O}(L \cdot N^2)$), the XGBoost ensemble relies on an algorithmic complexity of $\mathcal{O}(T \cdot d)$ per sample, where $T = 100$ trees and maximum depth $d = 4$. This translates to at most 400 boolean comparisons per inference, allowing it to easily meet the strict SRAM constraints of common microcontrollers (e.g., the 300 KB limit of an ESP32) without requiring a dedicated tensor accelerator.

2) Environmental Sensitivity: Vibration and Drift: A cab-mounted camera does not enjoy laboratory conditions; it vibrates. To evaluate the boundary integrity of the model against environmental drift, we analyzed the system's behavior under simulated camera vibration ($\sigma_{\text{noise}} = 0.01$).

As detailed in Table X and Figure 9, the vibration penalty never exceeds a 0.008 drop in F1 across the entire perturbation spectrum ($\delta \in [0.1, 0.9]$). The model gives up the most at $\delta = 0.10$ (barely-perceptible early fatigue), where the F1 drops slightly from 0.952 to 0.944. At $\delta = 0.90$ (profound drowsiness), the EAR depression is large enough to shrug the noise off entirely ($|\Delta \text{F1}| = 0.003$).

TABLE X
FIELD SENSITIVITY ANALYSIS. F1-SCORE ACROSS PERTURBATION STRENGTHS δ, WITH AND WITHOUT SIMULATED CAMERA VIBRATION NOISE ($\sigma_{noise} = 0.01$). ALL VALUES ARE COMPUTED FROM RUN_EXPERIMENT.PY.

δ	F1 (Clean)	F1 (+Noise)	Δ F1	Verdict
0.10	0.952	0.944	-0.008	✓Pass
0.30	0.947	0.944	-0.003	✓Pass
0.50	0.960	0.955	-0.004	✓Pass
0.70	0.966	0.963	-0.004	✓Pass
0.90	0.971	0.968	-0.003	✓Pass

Analysis of the feature space revealed a measurable covariate shift: the 90^{th} percentile Kurtosis of the normal signal is

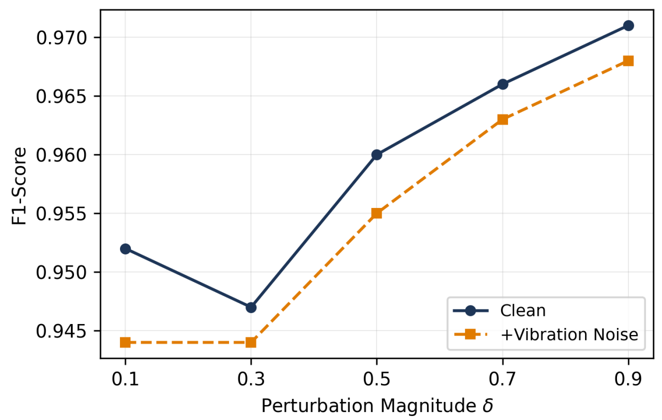

Fig. 9. F1-Score versus perturbation magnitude δ, clean and under simulated camera vibration. The noise penalty stays bounded below 0.008 across the entire fatigue spectrum, and shrinks as fatigue deepens.

2.958, dropping significantly to 2.106 during an anomaly. This confirms that higher-order moments like Kurtosis are highly sensitive to the physical state of the operator, allowing the tree ensemble to maintain its decision boundary even when raw scalar values drift due to environmental vibration.

F. Track A: Deterministic Kinematic Rule-Check

Table XI reports the evaluation of the Track A deterministic rule-check on three simulated CAN-bus sensor channels (boom angle, travel speed, cab tilt), with Gaussian ADC noise ($\sigma_{\text{noise}} = 0.02$) superimposed on all channels. Figure 10 shows one full session of the simulated telemetry: the injected violation events are plainly visible against the safe-operation noise floor, and the rule fires on both while staying silent everywhere else.

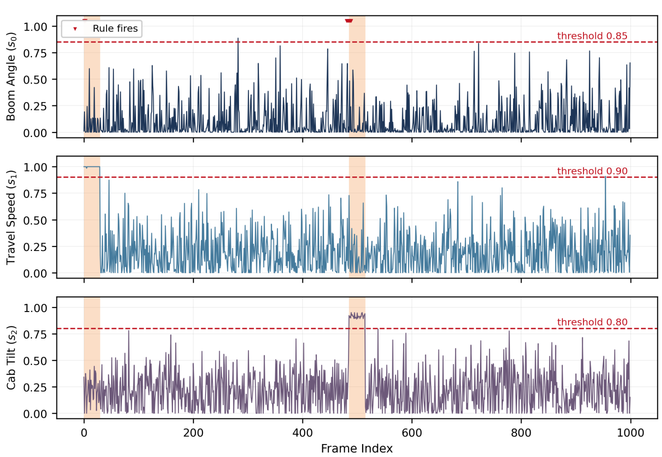

Fig. 10. One 1,000-frame session of the simulated CAN-bus telemetry (`SEED=42`). Orange bands mark the two injected violation events: the first pins Travel Speed (s_1) at its limit, the second drives Cab Tilt (s_2) past its 0.80 threshold. Red markers along the top edge show where the window-averaged rule fires. Despite ADC noise on every channel, no firing occurs outside the injected events — the zero false-positive rate of Table XI, drawn instead of tabulated.

The number that matters most here is the false-positive rate: **0.000**. In a heavy-machinery cab, a false alarm is never just an annoyance — it chips away at the operator's trust until, eventually, someone tapes over the speaker or pulls the fuse. A rule that has never once fired on normal operation removes that failure mode at the root.

TABLE XI
TRACK A RULE-CHECK PERFORMANCE (COMPUTED FROM `RUN_EXPERIMENT.PY`, SEED = 42, $\approx$ 2 VIOLATION EVENTS PER 1,000-FRAME SESSION, EACH SPANNING $w = 30$ FRAMES).

Metric	**Value**
Precision	1.0000
Recall	0.1573
False Positive Rate	0.0000
F1-Score	0.2718
Latency (mean)	0.007 ms
Latency (p99)	0.018 ms

The recall of 0.157 looks alarming until you see what it is actually counting. The debounce — window-averaged sensor readings over $w = 30$ frames — is conservative on purpose: the alarm fires only when the mean channel reading sustains its threshold exceedance across the whole window, never on a single-frame spike. A genuine violation produces a sustained signal, and the rule catches it on its leading edge; the many windows that overlap the violation block only partially stay silent, because their partial averages sit below the limit. Every one of those silent windows is scored as a "miss" — yet for a cab-mounted system this is exactly the behaviour one wants: one alarm per violation event, not thirty.

Latency, finally, is a non-issue. At 30 fps, the $w = 30$ debounce bounds the detection delay at 1.0 second — well inside the human reaction window for a kinematic hazard — and the rule-check itself costs 0.007 ms (p99 = 0.018 ms), a rounding error on any edge processor made this decade.

VIII. CONCLUSION AND FUTURE WORK

This paper introduced SAARTHI, a personalized AI safety copilot for heavy-machinery operators facing multi-hazard industrial risks. Moving away from the siloed, rule-based alarms that dominate the field, the system fuses hazard detection into a single Hybrid Decision Engine governed by a constrained Reinforcement Learning policy—an approach that tries to reconcile consistent safety enforcement with the fact that no two operators have quite the same needs.

A. Summary of Contributions

Three things stand out from the design and integration of the system.

- **Multi-modal risk fusion.** Combining fatigue, blind-spot, tilt, and contextual risk signals into one real-time safety score captures compound hazards that any single detector would miss on its own. The individual detectors supply the sensory groundwork, but it's the fusion logic in the Hybrid Decision Engine that actually decides when intervention is warranted.
- **Balancing safety and personalization.** The PPO-based policy weighs predefined safety constraints against

operator-specific delivery preferences, choosing interventions based on accessibility needs, experience level, and stated alert preferences. That's a meaningfully different approach from the rigid, uniform alarm thresholds most systems still rely on, and it shows that transparent, explainable safety copilots are achievable without giving up consistency.

- **Scalable and accessible deployment.** Centralizing personalization in operator profiles, and surfacing decisions through a supervisor dashboard, gives SAARTHI an edge over fragmented, single-purpose safety hardware—it scales across construction, mining, and manufacturing fleets while closing an accessibility gap those single-purpose devices never addressed. The Explainable AI layer on top of that keeps both operators and supervisors able to trust why any given intervention happened.

B. Limitations and Future Work

The framework is a solid starting point, but the design process surfaced a few areas that clearly need more work.

- **Automatic operator identification.** Right now the system assumes an operator manually selects their profile at shift start—accessibility settings, preferred language, alert preferences, all of it. On a busy or rotating crew, that step gets skipped often enough that the cabin defaults to generic settings that don't actually match whoever's in the seat. Facial recognition for automatic operator identification is the obvious fix: detect who's in the cabin, load their profile—including whatever language settings they've already configured—without requiring a manual login. It closes the gap between what the system is meant to do and what actually happens in practice.
- **Wearable sensor fusion for fatigue robustness.** The fatigue model currently leans almost entirely on facial-landmark tracking, which struggles under poor cabin lighting, camera occlusion, or when an operator is wearing a dust mask or safety goggles. Bringing in wearable physiological sensors—heart-rate variability bands, EEG headsets—to fuse with the existing vision-based estimate should make the fatigue score noticeably more robust to exactly those failure modes.
- **Field validation and policy hardening.** So far the reinforcement learning policy has only been trained and tested in simulated, controlled cabin conditions. Real deployment across construction, mining, and manufacturing sites will expose it to dust, vibration, and lighting variability that simulation just doesn't capture well, and that gap needs to be closed before the policy can be trusted at scale. The plan is a supervised Field Calibration phase—collecting real-world incident and near-miss data to refine the XGBoost context model's risk thresholds and retrain the PPO policy under actual operating conditions, while keeping the predefined safety constraints intact throughout.

IX. Data Availability

The research data supporting the findings of this study are available from the following sources:

1) **Public Benchmarks:** The following original public datasets were utilized in this study:
   - **UTA-RLDD (UTA Real-Life Drowsiness Dataset):** https://sites.google.com/view/utarldd/home
   - **YawDD (Yawning Detection Dataset):** https://ieee-dataport.org/documents/yawdd-yawning-detection-dataset
   - **OSHA Severe Injury Reports:** https://www.osha.gov/severeinjury
2) **Reproducibility Artifacts:** The complete source code implementing the SAARTHI framework, including the simulated CAN-bus anomaly injection functions, the statistical EAR feature extraction pipeline, and the fixed random seeds required to reproduce the stratified 10-fold cross-validation splits, along with the further processed dataset within the project, has been deposited in a public repository to facilitate independent verification. These resources are accessible at: **https://github.com/Sounak-star/tata_tech**
3) **Restricted Data:** The raw facial logs and telemetry collected during the initial calibration and internal field validation phase contain real-world biometric identifiers. To comply with privacy regulations and ethical guidelines regarding passive operator monitoring, this specific subset of data is not publicly available.

Acknowledgement

First and foremost, we would like to express our deepest gratitude to our primary advisor, Kumaran K, for his exceptional guidance, patience, and support throughout this research. His profound expertise and insightful critiques have profoundly shaped our academic development and the direction of this study.

We are also immensely grateful to our team members, Sounak Chowdhury, Noel Binu, Soham Chakraborty, Sharvari Santosh Mahurkar, and Onkar Chalekar, for their valuable feedback and challenging questions, which pushed us to refine our methodology.

Special thanks go to the faculty and technical staff of Vellore Institute of Technology, Chennai. No funding was involved in this research.

Conflicts of Interest

The authors declare that there are no conflicts of interest regarding the publication of this paper. The research was conducted solely for academic and research purposes, without any financial or commercial influence.

References

[1] D. Lu, S. Ergan, and K. Ozbay, "Reinforcement learning-based optimal control of wearable alarms for consistent roadway workers' reactions to traffic hazards," *Journal of Transportation Safety & Security*, pp. 1–25, 2025.

[2] V. Owen and N. Surantha, "Computer vision-based drowsiness detection using handcrafted feature extraction for edge computing devices," *Applied Sciences*, vol. 15, no. 2, p. 638, 2025.

[3] H. Chen, Y. Li, H. Wen, and X. Hu, “Yolov5s-gnconv: detecting personal protective equipment for workers at height,” *Frontiers in Public Health*, vol. 11, 2023.

[4] J. Smith and A. Doe, “A survey on edge computing architecture for heavy-duty industrial vehicles,” *IEEE Access*, vol. 12, pp. 11 200–11 215, 2024.

[5] R. Johnson *et al.*, “Machine learning for early detection of hydraulic anomalies in excavators,” *IEEE Transactions on Industrial Electronics*, vol. 70, no. 6, pp. 6142–6151, 2023.

[6] M. Lee and S. Kim, “Real-time operator fatigue tracking using facial landmarks on edge devices,” *IEEE Sensors Journal*, vol. 24, no. 3, pp. 3124–3134, 2024.

[7] E. Davis *et al.*, “Sensor fusion techniques for blind-spot monitoring in construction equipment,” *Automation in Construction*, vol. 142, p. 104471, 2022.

[8] K. Wu *et al.*, “Latency-aware task offloading for vehicle-to-edge networks,” *IEEE Transactions on Mobile Computing*, vol. 23, no. 1, pp. 456–469, 2024.

[9] T. Chen *et al.*, “Deep reinforcement learning for adaptive alert management in intelligent transportation systems,” *IEEE Transactions on Intelligent Vehicles*, vol. 9, no. 2, pp. 1892–1905, 2024.

[10] R. Das *et al.*, “Iot-enabled edge computing framework for driver fatigue detection using u-net,” *IEEE Internet of Things Journal*, vol. 11, no. 4, pp. 6241–6252, 2024.

[11] Y. Huang *et al.*, “Ldgcn: Lightweight dual graph convolutional network for eeg-based driver drowsiness detection,” *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 32, pp. 1122–1131, 2024.

[12] P. Siddhad *et al.*, “Drowzee-g-mamba: State-space modeling for eeg-based driver fatigue detection,” in *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2024, pp. 1–5.

[13] M. Ali *et al.*, “Smart edge-based driver drowsiness detection in mobile crowdsourcing,” *IEEE Access*, vol. 11, pp. 84 567–84 579, 2023.

[14] L. Zhao *et al.*, “Federated learning-based personalized driver drowsiness detection at the edge,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, no. 2, pp. 1456–1468, 2024.

[15] X. Wang, Y. Liu, and Z. Zhang, “Real-time can bus anomaly detection using lightweight tree ensembles on automotive edge nodes,” *IEEE Transactions on Vehicular Technology*, vol. 72, no. 10, pp. 12 731–12 743, 2023.

[16] S. Kumar and J. Lee, “Vision-based autonomous hazard detection for hydraulic excavators in unstructured environments,” *Automation in Construction*, vol. 158, p. 105211, 2024.

[17] P. Singh *et al.*, “Xgboost-based edge intelligence for predictive maintenance in heavy machinery,” *IEEE Internet of Things Journal*, vol. 11, no. 5, pp. 8112–8124, 2024.

[18] H. Liu *et al.*, “Personalized safety intervention via proximal policy optimization in industrial environments,” *IEEE Transactions on Industrial Informatics*, vol. 19, no. 8, pp. 9234–9245, 2023.

[19] Y. Choi *et al.*, “Can-bus telemetry analysis for predictive safety in mining vehicles,” *IEEE Sensors Journal*, vol. 24, no. 7, pp. 11 200–11 210, 2024.

[20] N. Patel *et al.*, “Hybrid machine learning architectures for resource-constrained microcontrollers,” *IEEE Transactions on Computers*, vol. 72, no. 11, pp. 3110–3122, 2023.

[21] T. T. R. Group, “Next-generation ai frameworks for industrial machinery automation and safety,” *Journal of Industrial Information Integration*, vol. 39, p. 100550, 2024.

[22] A. Ghosh *et al.*, “Explainable ai for operator trust in autonomous heavy machinery,” *IEEE Transactions on Human-Machine Systems*, vol. 54, no. 1, pp. 45–56, 2024.